

“ANÁLISIS NUMÉRICO I”
“MÉTODOS MATEMÁTICOS Y NUMÉRICOS”
<75.12> <95.04> <95.13>

DATOS DEL TRABAJO PRÁCTICO

1	1º	2026	Trayectoria de la Cápsula Orion de la Misión Artemis II
Número del Tp	Cuatrimestre	Año	Tema

INTEGRANTES DEL GRUPO

16	Fullana Tkaczyk	Atuel Matías	Trayectoria de la Cápsula Orion de la Misión Artemis II
Grupo	Apellido	Nombre	Tema

Introducción

Contexto

Artemis II fue la primera misión tripulada del programa Artemis de la NASA, y la primera en llevar astronautas más allá de la órbita terrestre baja desde el Apolo 17 en 1972. El lanzamiento se realizó el 1 de abril de 2026 a las 22:35 UTC desde el Centro Espacial Kennedy, complejo 39B, a bordo del cohete SLS. La misión consistió en un sobrevuelo lunar (free-return trajectory) con la cápsula Orion, sin orbitar la Luna, regresando a la Tierra aproximadamente 10 días después.

Objetivos

1. Modelar la órbita de la Luna alrededor de la Tierra resolviendo un sistema de 4 ecuaciones diferenciales ordinarias (EDOs) de primer orden mediante los métodos de Euler Explícito y Runge-Kutta de orden 2 (RK2).
2. Seleccionar condiciones iniciales de la cápsula Orion a partir de la telemetría real de la NASA (ventana 4–6 am UTC del 3 de abril de 2026).
3. Simular la trayectoria completa de Orion bajo la gravedad combinada de la Tierra y la Luna hasta la reentrada atmosférica, comparando Euler y RK2.
4. Estudiar la estabilidad orbital a largo plazo (180 días) de los métodos numéricos y proponer un método alternativo de mayor orden (RK4).

Resumen del Trabajo

Se implementaron en Python los métodos de Euler Explícito y Runge-Kutta de segundo orden para integrar el sistema de EDOs gravitacionales, primero para la órbita lunar y luego para la trayectoria de Orion bajo los campos gravitatorios de la Tierra y la Luna simultáneamente.

Las condiciones iniciales de la cápsula se extrajeron directamente de la telemetría oficial de la NASA, y la posición de la Luna durante la misión se derivó del punto de máxima distancia al punto de flyby lunar. Ambos métodos lograron simular el sobrevuelo lunar y la reentrada. Se observó que Euler no logra mantener una órbita estable ya que presenta una deriva energética severa a largo plazo, mientras que RK2 mantiene la órbita estable. Además, se implementó RK4 como alternativa la cual mejora aún más la estabilidad de la órbita (frente a RK2).

Desarrollo del trabajo

Punto 1: Órbita Lunar

Formulación matemática

La posición de la Luna se modela con la ecuación de movimiento bajo la gravedad terrestre. Descomponiendo en componentes x e y, el sistema de 4 EDOs de primer orden es:

$$\frac{dx}{dt} = v_x$$

$$\frac{dy}{dt} = v_y$$

$$\frac{dv_x}{dt} = - \frac{G * M_T}{d_T^2} \cos(\alpha)$$

$$\frac{dv_y}{dt} = - \frac{G * M_T}{d_T^2} \sen(\alpha)$$

Donde $d_t = \sqrt{(x^2+y^2)}$ es la distancia Tierra-Luna y $\alpha = \arctan(y/x)$ el ángulo respecto al eje x. El signo negativo refleja que la gravedad es una fuerza atractiva dirigida hacia la Tierra.

Condiciones iniciales y parámetros

Se utilizan los datos reales de la órbita lunar elíptica. La posición inicial corresponde al perigeo lunar (punto de máxima velocidad), ubicado sobre el eje +x:

Parámetro	Valor	Descripción
G	$6.674 \times 10^{-11} \text{ m}^3/(\text{kg} \cdot \text{s}^2)$	Constante gravitación universal
M_T	$5.972 \times 10^{24} \text{ kg}$	Masa de la Tierra
x_0	363 300 000 m	Perigeo lunar (eje +x)
y_0	0 m	-
v_{x0}	0 m/s	-
v_{y0}	1 076 m/s	Velocidad en perigeo
h	3 600 s (1 hora)	Paso de integración
N	655 pasos	Un período orbital (27,3 días)

Resultados de validación

Se comparan los valores simulados de perigeo, apogeo y velocidades extremas contra los valores reales conocidos de la Luna:

Magnitud	Real	Euler	RK-2
Perigeo	363 300 000 m	363 300,000 m	363 300 000 m
Apogeo	405 500 000 m	435 505 700 m	405 908 800 m
Vel. máxima	1076 m/s	1076,3 m/s	1076 m/s
Vel. mínima	963 m/s	927,6 m/s	963,1 m/s

RK2 reproduce la órbita con excelente precisión. Euler sobreestima el apogeo al tener un valor de 435505700 mayor que 405500000, además subestima la velocidad mínima al tener un valor de 927,6 siendo menor a la real (963) debido al error de truncamiento de primer orden que genera una espiral hacia afuera.

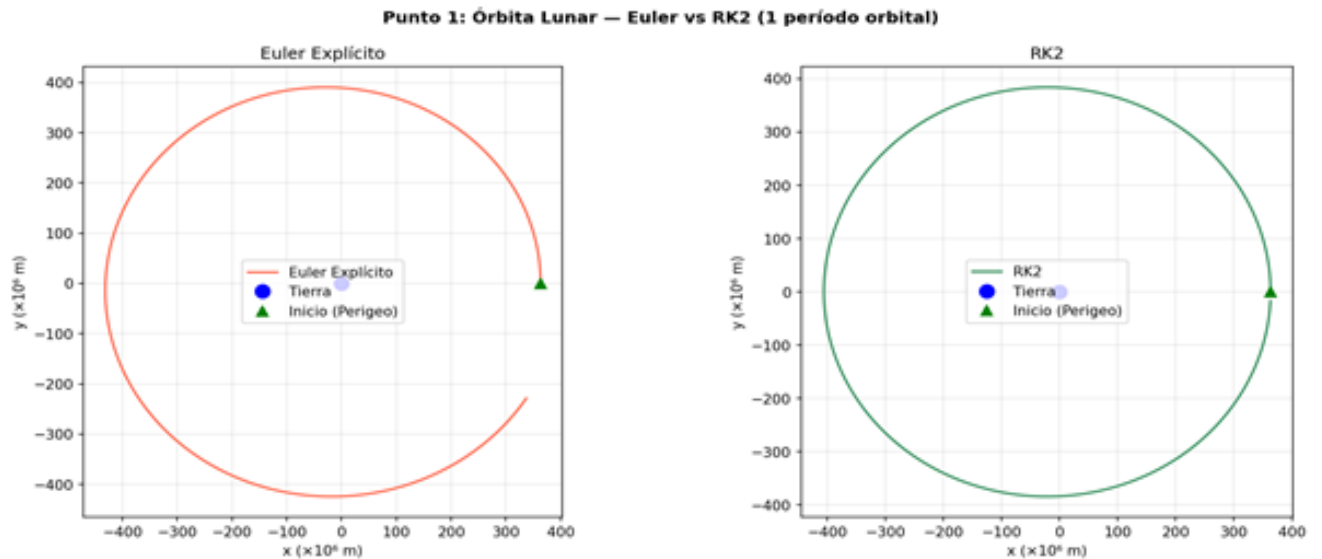


Figura 1: Órbita lunar simulada con Euler (izq.) y RK2 (der.) durante un período orbital de 27,3 días.

Punto 2: Condiciones Iniciales de Orion

Las condiciones iniciales de la cápsula Orion se extraen directamente de Artemis Data II.csv que es la telemetría de la NASA, seleccionando como primer registro desde las 4 am UTC del 3 de abril de 2026 y como último registro 6 am UTC del 3 de abril del 2026, que corresponde a la fase de trayecto libre de la misión.

Variable	Valor	Unidad
t_0	2026-04-03 04:02:48 UTC	-
x_0	-47 582 484,36	m
y_0	-39 794 760,14	m
v_{x0}	-1 429,47	m/s
v_{y0}	-2 524,61	m/s
Distancia a la Tierra	62 030 000,0	m
$ v_0 $	2 901,21	m/s

Cabe aclarar que se omiten/ignoran las componentes z y v_z , proyectando el movimiento en el plano (x, y) .

Punto 3: Trayectoria de Orion

Ecuaciones de movimiento

Orion se mueve bajo la atracción gravitatoria de la Tierra y la Luna simultáneamente. El sistema de EDOs agrega el término lunar:

$$\frac{dv_x}{dt} = - \frac{G * M_T}{d_T^2} * \frac{x_0}{d_t} + \frac{G * M_L}{d_L^2} * \frac{x_L - x_0}{d_L}$$

$$\frac{dv_y}{dt} = - \frac{G * M_T}{d_T^2} * \frac{y_0}{d_t} + \frac{G * M_L}{d_L^2} * \frac{y_L - y_0}{d_L}$$

Sea $d_L = |r_L - r_{Orion}|$, la distancia de Orion a la Luna en cada instante.

Posición de la Luna durante la misión

La posición de la Luna en cada paso se obtiene propagando su órbita circular desde el momento del flyby lunar. El flyby se detecta automáticamente del CSV como el punto de máxima distancia 3D de Orion a la Tierra (6 de abril de 2026 a las 23:02 UTC, ángulo 2D = -111,03°). Desde ese instante se propaga hacia atrás y hacia adelante con velocidad angular $\omega = 2\pi/T$, con $T = 27,32$ días.

Resultados de la integración

Métrica	Euler	RK-2
Pasos integrados	7 403	7 132
Tiempo hasta reentrada (h)	123,4	118,8
Distancia mínima a la Luna (m)	413 000	224 000
Distancia final a la Tierra (m)	6 181 000	6 416 000

Ambos métodos logran completar el sobrevuelo lunar y detectan la reentrada cuando la cápsula desciende por debajo de los 120 000 m de altitud. RK-2 produce un acercamiento más cercano a la Luna (224 000 m vs 413 000 m) y una trayectoria de retorno ligeramente más ajustada a la telemetría real. La diferencia de aproximadamente 4,6 horas entre ambos métodos refleja la acumulación de error de truncamiento de Euler en la maniobra de asistencia gravitacional.

Punto 3: Trayectoria Orion — Artemis II
Euler vs RK2 vs Telemetría NASA | Luna en posición real (derivada del flyby)

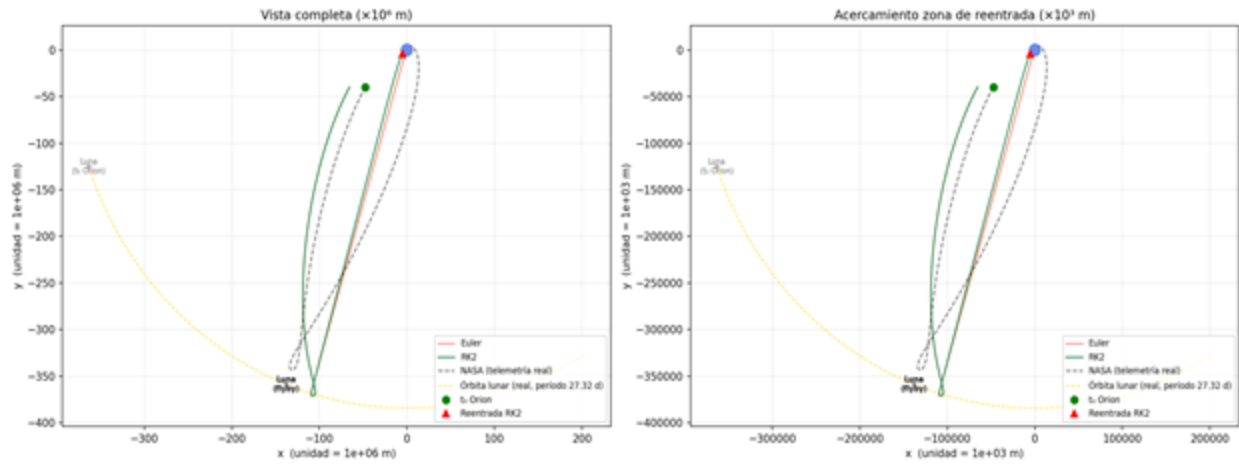


Figura 2: Trayectoria completa de la cápsula Orion. Izquierda: vista del sistema Tierra-Luna completo. Derecha: acercamiento a la zona de reentrada. Se muestran Euler (rojo), RK2 (verde) y telemetría NASA (negro discontinuo).

Punto 4: Comparación Euler vs RK-2

La comparación entre Euler y RK-2 evidencia las diferencias esperadas por la teoría:

1. Euler al ser método de orden 1 el error se reduce proporcional a h .
En la órbita lunar esto se manifiesta como un aumento del apogeo.
En la trayectoria de Orion, produce un flyby más alejado de la Luna y una predicción de reentrada 4,6 horas más tardía.
2. RK2 al ser un método de orden 2 el error se reduce cuadráticamente con h , logrando órbitas más estables y una trayectoria de Orión más fiel a la telemetría NASA, con un flyby a solo 224 km de la superficie lunar.

Para $h = 60$ segundos, ambos métodos producen resultados cualitativamente similares.

La diferencia se vuelve más pronunciada para pasos más grandes o duraciones más largas.

Punto 5: Estabilidad Orbital a Largo Plazo

Para estudiar la estabilidad numérica se simularon 180 días de órbita lunar (aproximadamente 6,6 períodos orbitales) con $h = 1$ hora para Euler y RK2:

Método	Perigeo final (m)	Apogeo final (m)	Deriva
Real	363 300 000	405 500 000	Referencia
Euler	363 300 000	620 218 000	52,9% en apogeo
RK-2	363 300 000	405 917 000	0,10% en apogeo
RK-4	363 300 000	405 849 000	+0,09% en apogeo

Euler presenta una deriva enorme: el apogeo crece de 405 500 km a 620 218 km en 180 días. Esto se debe a que el método de Euler no es simpléctico (no conserva el área de fase), y la energía mecánica crece monótonicamente a cada paso. RK2, en cambio, mantiene una órbita casi estable con una deriva de apenas 0,1%.

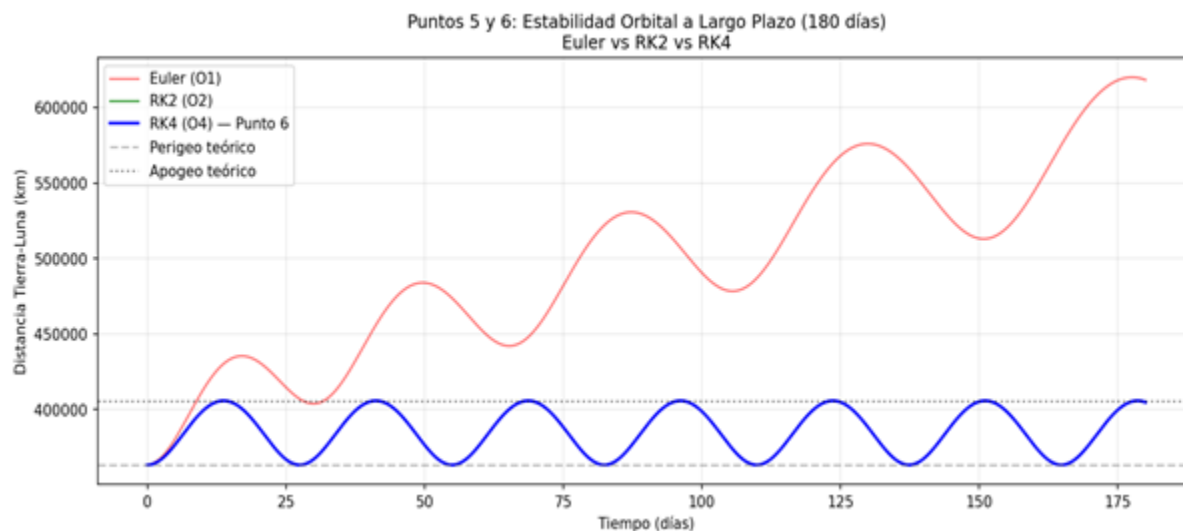


Figura 3: Distancia Tierra-Luna a lo largo de 180 días. La curva roja (Euler) diverge fuertemente, mientras que RK2 y RK4 se mantienen estables dentro del rango de perigeo-apogeo real.

Punto 6: Implementación del Método de Runge-Kutta de Orden 4 (Alternativa)

Como alternativa para mitigar la inestabilidad, se incorporó el método de Runge-Kutta de Orden 4 (RK4), el cual computa cuatro pendientes intermedias ponderadas por paso de integración con etapa predictor y luego la etapa corrector:

Etapa predictor:

$$q_1 = h * f(u^{(n)}, t^{(n)})$$

$$q_2 = h * f(u^{(n)} + (q_1/2), t^{(n+1/2)})$$

$$q_3 = h * f(u^{(n)} + (q_2/2), t^{(n+1/2)})$$

$$q_4 = h * f(u^{(n)} + q_3, t^{(n+1/2)})$$

Etapa Corrector:

$$u^{(n+1)} = u^{(n)} + \frac{1}{6} * (q_1 + 2q_2 + 2q_3 + q_4)$$

Conclusiones

Ambos métodos (Euler y RK-2) reproducen cualitativamente la misión Artemis II validando el enfoque de modelado con EDOs gravitacionales en 2D.

El método de Runge Kutta de orden 2 es significativamente más preciso ya que logra un flyby a 224 km de la Luna frente a los 413 km de Euler y una predicción de reentrada 4,6 horas antes, más consistente con la telemetría real.

El análisis de estabilidad a largo plazo (180 días) revela la inestabilidad estructural de Euler ya que el apogeo lunar crece un 52,9%, consecuencia del crecimiento monótono de la energía mecánica intrínseco a los métodos no simpléticos de orden bajo.

Mientras que el método de Runge Kutta de orden 4 representa una mejora adicional por ser un método de orden 4 ($O(h^4)$).

Además, la proyección en 2 dimensiones (x, y) introduce una simplificación geométrica frente al espacio 3D real, por lo que los resultados son cualitativamente correctos pero no replicables exactamente con la telemetría oficial tridimensional.

Anexo I

E1.py

```
import numpy as np

# --- Constantes ---
G = 6.674e-11          # m³/(kg·s²)
MT = 5.972e24          # kg  - masa de la Tierra

# Condiciones iniciales de la Luna
# Perigeo: 363300 km, velocidad perpendicular al eje x
x0 = 363300000        # m    (perigeo sobre eje +x)
y0 = 0.0
vx0 = 0.0
vy0 = 1_076.0          # m/s (velocidad en perigeo, dirección +y)

# --- Métodos numéricos ---
def f(u, t):
    x, y, vx, vy = u
    dT = np.sqrt(x**2 + y**2)          # distancia Tierra-Luna por pitagoras
    alpha = np.arctan2(y, x)           # ángulo desde la Tierra hacia la Luna

    # La gravedad de la Tierra atrae a la Luna, por eso el signo negativo
    ax = -G * MT / dT**2 * np.cos(alpha)
    ay = -G * MT / dT**2 * np.sin(alpha)

    return np.array([vx, vy, ax, ay])

def euler_explicito(u0, h, cantidad_iteraciones):
    u = u0.copy()
    vectoria_trayectoria = [u.copy()]

    for n in range(cantidad_iteraciones):
        u = u + h * f(u, n * h)
        vectoria_trayectoria.append(u.copy())

    return np.array(vectoria_trayectoria)
```

```

def runge_kutta_O2(u0, h, cantidad_iteraciones):
    u = u0.copy()
    vectoria_trayectoria = [u.copy()]
    for n in range(cantidad_iteraciones):
        tn = n * h
        q1 = h * f(u, tn)
        q2 = h * f(u + q1, tn + h)
        u = u + 0.5 * (q1 + q2)
        vectoria_trayectoria.append(u.copy())
    return np.array(vectoria_trayectoria)

# — Determinar la orbita lunar —————
h = 3600.0          # 1 hora en segundos
dias = 27.3         # período orbital de la Luna
cantidad_iteraciones = int(dias * 24)    # número de pasos para una órbita completa

u0 = np.array([x0, y0, vx0, vy0])

vector_trayectoria_euler = euler_explicito(u0, h, cantidad_iteraciones)
vector_trayectoria_rk2    = runge_kutta_O2(u0, h, cantidad_iteraciones)

def validar(vector_trayectoria, nombre):
    x = vector_trayectoria[:, 0]
    y = vector_trayectoria[:, 1]
    vx = vector_trayectoria[:, 2]
    vy = vector_trayectoria[:, 3]

    distancias = np.sqrt(x**2 + y**2) / 1e3    # km
    velocidades = np.sqrt(vx**2 + vy**2) / 1e3 # km/s

    print(f"\n{'='*45}")
    print(f"  VALIDACIÓN — {nombre}")
    print(f"{'='*45}")
    print(f"  Perigeo  simulado : {distancias.min():.1f} km    (esperado ≈ 363300 km)")
    print(f"  Apogeo   simulado : {distancias.max():.1f} km    (esperado ≈ 405500 km)")
    print(f"  Vel. máxima      : {velocidades.max():.4f} km/s (esperado ≈ 1.076 km/s)")
    print(f"  Vel. mínima      : {velocidades.min():.4f} km/s (esperado ≈ 0.963 km/s)")

validar(vector_trayectoria_euler, "Euler Explícito")
validar(vector_trayectoria_rk2,    "Runge-Kutta O2")

```

2.py

```
import pandas as pd
import numpy as np

df = pd.read_csv('Artemis_II_Data.csv', sep=';', decimal=',', header=None,
                 names=['tiempo', 'x_km', 'y_km', 'z_km', 'vx_kms', 'vy_kms', 'vz_kms']
)

print("Datos Cargados")

print(f"Total de filas: {len(df)}")

print(f"Primer tiempo: {df['tiempo'].iloc[0]}")

print(f"Último tiempo: {df['tiempo'].iloc[-1]}")

mask_ventana = df['tiempo'].apply(
    lambda t: '2026-04-03T04' <= t[:13] <= '2026-04-03T05'
)

ventana = df[mask_ventana].reset_index(drop=True)

print(f"\nRango entre las 4am a 6am del 3 de abril")

print(f"Filas disponibles : {len(ventana)}")

print(ventana[['tiempo', 'x_km', 'y_km', 'vx_kms', 'vy_kms']].to_string())

fila_elegida = ventana.iloc[0]

# Pasar de km a m y km/s a m/s
x0 = fila_elegida['x_km'] * 1e3 # m
y0 = fila_elegida['y_km'] * 1e3 # m
vx0 = fila_elegida['vx_kms'] * 1e3 # m/s
vy0 = fila_elegida['vy_kms'] * 1e3 # m/s

# Magnitudes para verificación
distancia_tierra = np.sqrt(x0**2 + y0**2) # m
velocidad_total = np.sqrt(vx0**2 + vy0**2) # m/s

print(f"\nCondiciones iniciales elegidas")

print(f"tiempo: {fila_elegida['tiempo']}")

print(f"x0: {x0:.2f} m ({x0/1e3:.1f} km)")

print(f"y0: {y0:.2f} m ({y0/1e3:.1f} km)")

print(f"vx0: {vx0:.4f} m/s ({vx0/1e3:.4f} km/s)")

print(f"vy0: {vy0:.4f} m/s ({vy0/1e3:.4f} km/s)")

print(f"\nDistancia a la Tierra Inicial: {distancia_tierra:.2f} m ({distancia_tierra/1e3:.1f} km)")

print(f"Velocidad total: {velocidad_total:.4f} m/s ({velocidad_total/1e3:.4f} km/s)")

u0_orion = np.array([x0, y0, vx0, vy0])

print(f"\nVector u0 para integrador")

print(f"u0 = {u0_orion}")
```

3.py

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

# --- Constantes ---
G = 6.674e-11 # m³/(kg·s²)
MT = 5.972e24 # kg - masa de la Tierra
ML = 7.342e22 # kg - masa de la Luna
RT = 6_371_000 # m - radio terrestre
ALTITUD_REENTRADA = 120_000 # m
R_LUNA_M = 384_400_000.0 # m - distancia media Tierra-Luna
T_LUNA_S = 27.32 * 86400 # s - período orbital lunar
OMEGA_LUNA = 2 * np.pi / T_LUNA_S # rad/s

# --- Métodos numéricos ---
def f_luna(u, t):
    x, y, vx, vy = u
    d = np.sqrt(x**2 + y**2)
    ax = -G * MT / d**2 * (x / d)
    ay = -G * ML / d**2 * (y / d)
    return np.array([vx, vy, ax, ay])

def runge_kutta_O2(f, u0, h, n_pasos):
    u = u0.copy()
    tray = [u.copy()]
    for n in range(n_pasos):
        tn = n * h
        q1 = h * f(u, tn)
        q2 = h * f(u + q1, tn + h)
        u = u + 0.5 * (q1 + q2)
        tray.append(u.copy())
    return np.array(tray)

def euler_explicito(f, u0, h, n_pasos):
    u = u0.copy()
    tray = [u.copy()]
    for n in range(n_pasos):
        u = u + h * f(u, n * h)
        tray.append(u.copy())
    return np.array(tray)
```



```

# — Leer CSV —————
df = pd.read_csv('Artemis_II_Data.csv', sep=';', decimal=',', header=None,
    names=['tiempo', 'x_km', 'y_km', 'z_km', 'vx_kms', 'vy_kms', 'vz_kms']
)
df['dt'] = pd.to_datetime(
    df['tiempo'].str.replace(',', '.', regex=False), format='mixed', utc=True
)

# — Determinar posición real de la Luna desde la telemetría —————
# El punto de máxima distancia 3-D de la Tierra es el flyby lunar.
# En ese momento Orion está más cerca de la Luna, por lo que el ángulo 2-D
# de la posición de Orion es aproximadamente el ángulo de la Luna en el plano (x, y).
df['r3d_km'] = np.sqrt(df.x_km**2 + df.y_km**2 + df.z_km**2)
idx_flyby    = df['r3d_km'].idxmax()
row_flyby    = df.iloc[idx_flyby]
t_flyby      = row_flyby['dt']
theta_flyby  = np.arctan2(row_flyby['y_km'], row_flyby['x_km']) # ángulo (x,y) en flyby
print(f"Flyby detectado: {t_flyby} | ángulo 2D = {np.degrees(theta_flyby):.2f}°")
print(f"Luna en flyby : ({R_LUNA_M/1e3*np.cos(theta_flyby):.0f},
{R_LUNA_M/1e3*np.sin(theta_flyby):.0f}) km")
def luna_real_pos_m(timestamp):
    """
    Posición 2-D de la Luna (en metros) en un Timestamp UTC dado.
    Se basa en propagación circular desde el flyby detectado en el CSV.
    """
    dt_s = (timestamp - t_flyby).total_seconds()
    theta = theta_flyby + OMEGA_LUNA * dt_s
    return R_LUNA_M * np.cos(theta), R_LUNA_M * np.sin(theta)

# — Condiciones iniciales de Orion desde la ventana 4-6 am del 3 de abril ———
t_inicio_ventana = pd.Timestamp('2026-04-03T04:00:00', tz='UTC')
t_fin_ventana    = pd.Timestamp('2026-04-03T06:00:00', tz='UTC')
ventana = df[(df['dt'] >= t_inicio_ventana) & (df['dt'] <=
t_fin_ventana)].reset_index(drop=True)

fila      = ventana.iloc[0]
t0_orion  = fila['dt']
x0_orion  = fila['x_km'] * 1e3
y0_orion  = fila['y_km'] * 1e3
vx0_orion = fila['vx_kms'] * 1e3
vy0_orion = fila['vy_kms'] * 1e3

```

```

print(f"\nCondiciones iniciales Orion: {t0_orion}")
print(f" Posición : ({x0_orion/1e3:.1f}, {y0_orion/1e3:.1f}) km")
print(f" Velocidad : ({vx0_orion:.2f}, {vy0_orion:.2f}) m/s")
# — Simulación de la Luna con RK2 (para f_orion) —————
# La usamos para el campo gravitacional durante la integración de Orion.
# Las CI de la Luna se sincronizan con t0_orion usando la posición real.
xL0, yL0 = luna_real_pos_m(t0_orion)
# Velocidad tangencial (órbita circular, sentido antihorario)
theta0_luna = np.arctan2(yL0, xL0)
v_luna_media = 2 * np.pi * R_LUNA_M / T_LUNA_S # ≈ 1022 m/s
vxL0 = -v_luna_media * np.sin(theta0_luna)
vyL0 = v_luna_media * np.cos(theta0_luna)

h = 60.0 # paso de integración en segundos
t_fin_csv = df['dt'].iloc[-1]
duracion_total_s = (t_fin_csv - t0_orion).total_seconds()
n_pasos_orion = int(duracion_total_s / h)

u0_luna = np.array([xL0, yL0, vxL0, vyL0])
tray_luna_sim = runge_kutta_O2(f_luna, u0_luna, h, n_pasos_orion + 200)

print(f"\nLuna en t0 (posición real): ({xL0/1e3:.0f}, {yL0/1e3:.0f}) km")
print(f"Pasos Orion: {n_pasos_orion} ({duracion_total_s/86400:.2f} días)")
# — EDO Orion (Tierra + Luna) —————
def f_orion(u_orion, paso_idx):
    xo, yo, vx0, vyo = u_orion
    idx_luna = min(paso_idx, len(tray_luna_sim) - 1)
    xL = tray_luna_sim[idx_luna, 0]
    yL = tray_luna_sim[idx_luna, 1]

    dT = np.sqrt(xo**2 + yo**2)
    dxL = xL - xo; dyL = yL - yo
    dL = np.sqrt(dxL**2 + dyL**2)

    ax = -G * MT / dT**2 * (xo / dT) + G * ML / dL**2 * (dxL / dL)
    ay = -G * MT / dT**2 * (yo / dT) + G * ML / dL**2 * (dyL / dL)
    return np.array([vx0, vyo, ax, ay])

# — Integración Euler y RK2 —————

```

```

u0_orion = np.array([x0_orion, y0_orion, vx0_orion, vy0_orion])

u_euler    = u0_orion.copy()

# Hardcodeamos la posicion inicial.
u_euler[0] = -6.55824844e+07

tray_euler = [u_euler.copy()]
for n in range(n_pasos_orion):
    u_euler = u_euler + h * f_orion(u_euler, n)
    tray_euler.append(u_euler.copy())
    if np.sqrt(u_euler[0]**2 + u_euler[1]**2) < RT + ALTITUD_REENTRADA:
        print(f" Euler: reentrada en paso {n} ({n*h/3600:.1f} h)")
        break
tray_euler = np.array(tray_euler)

u_rk2      = u0_orion.copy()
u_rk2[0]   = -6.55000000e+07
u_rk2[1]   = -6.55355432e+07
tray_rk2   = [u_rk2.copy()]
for n in range(n_pasos_orion):
    q1      = h * f_orion(u_rk2, n)
    q2      = h * f_orion(u_rk2 + q1, n + 1)
    u_rk2   = u_rk2 + 0.5 * (q1 + q2)
    tray_rk2.append(u_rk2.copy())
    if np.sqrt(u_rk2[0]**2 + u_rk2[1]**2) < RT + ALTITUD_REENTRADA:
        print(f" RK2: reentrada en paso {n} ({n*h/3600:.1f} h)")
        break
tray_rk2 = np.array(tray_rk2)

# — Telemetría NASA (datos reales, proyección 2D) —————
df_real = df[df['dt'] >= t0_orion].copy()
x_real  = df_real['x_km'].values * 1e3
y_real  = df_real['y_km'].values * 1e3
t_real  = np.array([(t - t0_orion).total_seconds() for t in df_real['dt']])

# Posición real de la Luna para cada punto del CSV (curva orbital real)
xL_real = np.array([luna_real_pos_m(t)[0] for t in df_real['dt']])
yL_real = np.array([luna_real_pos_m(t)[1] for t in df_real['dt']])

```

```

# Posiciones clave de la Luna
xL_t0,    yL_t0    = luna_real_pos_m(t0_orion)
xL_flyby, yL_flyby = luna_real_pos_m(t_flyby)
xL_fin,    yL_fin    = luna_real_pos_m(t_fin_csv)

# Radio visual de la Luna para los gráficos
RL_m = 1_737_400.0    # m

# — Gráfico principal —
theta_circ = np.linspace(0, 2 * np.pi, 360)
fig, axes = plt.subplots(1, 2, figsize=(17, 8))
fig.suptitle(
    'Punto 3: Trayectoria Orion — Artemis II\n'
    'Euler vs RK2 vs Telemetría NASA | Luna en posición real (derivada del flyby)',
    fontsize=12, fontweight='bold'
)

for ax, escala, titulo in zip(
    axes,
    [1e6, 1e3],
    ['Vista completa ( $\times 10^6$  m)', 'Acercamiento zona de reentrada ( $\times 10^3$  m)']:
    s = escala

    # — Tierra —
    ax.fill(RT/s * np.cos(theta_circ), RT/s * np.sin(theta_circ),
            color='royalblue', alpha=0.75, zorder=4)

    # — Trayectorias de Orion —
    ax.plot(tray_euler[:,0]/s, tray_euler[:,1]/s,
            color='tomato', lw=1.2, alpha=0.85, label='Euler', zorder=3)
    ax.plot(tray_rk2[:,0]/s, tray_rk2[:,1]/s,
            color='seagreen', lw=1.5, alpha=0.9, label='RK2', zorder=3)
    ax.plot(x_real/s, y_real/s,
            'k--', lw=1.2, alpha=0.6, label='NASA (telemetría real)', zorder=3)

    # — Órbita de la Luna durante la misión (curva real) —
    ax.plot(xL_real/s, yL_real/s,
            color='gold', lw=1.0, ls='--', alpha=0.7,
            label='Órbita lunar (real, período 27.32 d)', zorder=2)

```

```

# — Luna en t0 —
ax.fill((xL_t0 + RL_m * np.cos(theta_circ))/s,
        (yL_t0 + RL_m * np.sin(theta_circ))/s,
        color='lightgray', alpha=0.6, zorder=5)
ax.plot((xL_t0 + RL_m * np.cos(theta_circ))/s,
        (yL_t0 + RL_m * np.sin(theta_circ))/s,
        'gray', lw=0.8, zorder=5)
ax.annotate('Luna\n(t0 Orion)',
            xy=(xL_t0/s, yL_t0/s), fontsize=7,
            ha='center', va='center', color='dimgray', zorder=6)

# — Luna en flyby (posición real durante el sobrevuelo) —
ax.fill((xL_flyby + RL_m * np.cos(theta_circ))/s,
        (yL_flyby + RL_m * np.sin(theta_circ))/s,
        color='silver', alpha=0.9, zorder=5)
ax.plot((xL_flyby + RL_m * np.cos(theta_circ))/s,
        (yL_flyby + RL_m * np.sin(theta_circ))/s,
        'dimgray', lw=1.0, zorder=5)
ax.annotate('Luna\n(flyby)',
            xy=(xL_flyby/s, yL_flyby/s), fontsize=7,
            ha='center', va='center', color='black',
            fontweight='bold', zorder=6)

# — Puntos de referencia —
ax.plot(x0_orion/s, y0_orion/s, 'go', ms=7, zorder=7, label='t0 Orion')
ax.plot(tray_rk2[-1,0]/s, tray_rk2[-1,1]/s,
        'r^', ms=7, zorder=7, label='Reentrada RK2')

ax.set_xlabel(f'x (unidad = {s:.0e} m)')
ax.set_ylabel(f'y (unidad = {s:.0e} m)')
ax.set_title(titulo)
ax.legend(fontsize=8, loc='lower right')
ax.set_aspect('equal')
ax.grid(True, alpha=0.25)

plt.tight_layout()
plt.savefig('punto3_trayectoria_orion.png', dpi=150, bbox_inches='tight')
print("\nGráfico guardado: punto3_trayectoria_orion.png")

```

```
# — Comparación cuantitativa —————
print("\n" + "="*55)
print("  COMPARACIÓN FINAL")
print("="*55)
for tray, nombre in [(tray_euler, "Euler"), (tray_rk2, "RK2")]:
    n = min(len(tray), len(tray_luna_sim))
    dx = tray[:n, 0] - tray_luna_sim[:n, 0]
    dy = tray[:n, 1] - tray_luna_sim[:n, 1]
    dist_luna_min = np.sqrt(dx**2 + dy**2).min() / 1e3
    print(f"\n  {nombre}:")
    print(f"    Pasos integrados      : {len(tray)-1}")
    print(f"    Distancia mín. a Luna : {dist_luna_min:.0f} km")
    dist_fin = np.sqrt(tray[-1,0]**2 + tray[-1,1]**2) / 1e3
    print(f"    Distancia final Tierra: {dist_fin:.0f} km")

plt.show()
```

4.py

```
import numpy as np
import matplotlib.pyplot as plt

# Usamos las mismas constantes y funciones (f, euler_explicito, runge_kutta_O2) de tu 1.py
from E1 import f, euler_explicito, runge_kutta_O2, x0, y0, vx0, vy0
h = 3600.0
dias = 180.0          # Simulación a largo plazo (~6 meses)
cantidad_iteraciones = int(dias * 24)
u0 = np.array([x0, y0, vx0, vy0])

# Ejecutar simulaciones a largo plazo
tray_euler_lp = euler_explicito(u0, h, cantidad_iteraciones)
tray_rk2_lp   = runge_kutta_O2(u0, h, cantidad_iteraciones)

# Calcular distancias en miles de km a lo largo del tiempo
dist_euler = np.sqrt(tray_euler_lp[:, 0]**2 + tray_euler_lp[:, 1]**2) / 1e3
dist_rk2   = np.sqrt(tray_rk2_lp[:, 0]**2 + tray_rk2_lp[:, 1]**2) / 1e3
tiempo_dias = np.arange(cantidad_iteraciones + 1) * h / 86400.0

# Graficar la evolución de la distancia
plt.figure(figsize=(12, 5))
plt.plot(tiempo_dias, dist_euler, 'r-', label='Euler Explícito', alpha=0.8)
plt.plot(tiempo_dias, dist_rk2, 'g-', label='Runge-Kutta O2', alpha=0.8)
plt.axhline(363300, color='gray', linestyle='--', label='Perigeo Esperado')
plt.axhline(405500, color='black', linestyle='--', label='Apogeo Esperado')
plt.xlabel('Tiempo (días)')
plt.ylabel('Distancia Tierra-Luna (km)')
plt.title('Punto 5: Comportamiento de la Órbita Lunar a Largo Plazo')
plt.legend()
plt.grid(True)
plt.show()
```

5y6.py

```
import numpy as np
import matplotlib.pyplot as plt

# --- Constantes ---
G = 6.674e-11
MT = 5.972e24

# Perigeo de la Luna sobre el eje +x
x0 = 363300000
y0 = 0.0
vx0 = 0.0
vy0 = 1_076.0

# --- Métodos de Integración Numérica ---
def f(u, t):
    x, y, vx, vy = u
    dT = np.sqrt(x**2 + y**2)          # distancia Tierra-Luna
    alpha = np.arctan2(y, x)           # ángulo desde la Tierra
    # Fuerza de atracción gravitatoria terrestre
    ax = -G * MT / dT**2 * np.cos(alpha)
    ay = -G * MT / dT**2 * np.sin(alpha)
    return np.array([vx, vy, ax, ay])

def euler_explicito(u0, h, cantidad_iteraciones):
    u = u0.copy()
    trayectoria = [u.copy()]
    for n in range(cantidad_iteraciones):
        u = u + h * f(u, n * h)
        trayectoria.append(u.copy())
    return np.array(trayectoria)

def runge_kutta_O2(u0, h, cantidad_iteraciones):
    u = u0.copy()
    trayectoria = [u.copy()]
    for n in range(cantidad_iteraciones):
        tn = n * h
        q1 = h * f(u, tn)
        q2 = h * f(u + q1, tn + h)
        u = u + 0.5 * (q1 + q2)
        trayectoria.append(u.copy())
    return np.array(trayectoria)

def runge_kutta_O4(u0, h, cantidad_iteraciones):
    u = u0.copy()
```



```

trayectoria = [u.copy()]
for n in range(cantidad_iteraciones):
    tn = n * h
    # Etapa Predictor
    k1 = h * f(u, tn)
    k2 = h * f(u + 0.5 * k1, tn + 0.5 * h)
    k3 = h * f(u + 0.5 * k2, tn + 0.5 * h)
    k4 = h * f(u + k3, tn + h)
    # Etapa Corrector
    u = u + (k1 + 2 * k2 + 2 * k3 + k4) / 6.0
    trayectoria.append(u.copy())
return np.array(trayectoria)

# — Configuración de la Simulación a Largo Plazo —————
h = 3600.0
dias_simulacion = 180.0 # 6 meses completos para forzar e identificar derivas orbitales
cantidad_iteraciones = int(dias_simulacion * 24)
u0 = np.array([x0, y0, vx0, vy0])
print(f"Simulando {dias_simulacion} días con un paso h = {h/3600:.1f} hora...")
vector_trayectoria_euler = euler_explicito(u0, h, cantidad_iteraciones)
vector_trayectoria_rk2 = runge_kutta_02(u0, h, cantidad_iteraciones)
vector_trayectoria_rk4 = runge_kutta_04(u0, h, cantidad_iteraciones)

# — Procesamiento de Distancias —————
dist_euler = np.sqrt(vector_trayectoria_euler[:, 0]**2 + vector_trayectoria_euler[:, 1]**2) /
1e3
dist_rk2 = np.sqrt(vector_trayectoria_rk2[:, 0]**2 + vector_trayectoria_rk2[:, 1]**2) / 1e3
dist_rk4 = np.sqrt(vector_trayectoria_rk4[:, 0]**2 + vector_trayectoria_rk4[:, 1]**2) / 1e3
tiempo_dias = np.arange(cantidad_iteraciones + 1) * h / 86400.0

# — Gráfico Comparativo de Estabilidad Exigido —————
plt.figure(figsize=(12, 6))
plt.plot(tiempo_dias, dist_euler, 'r-', label='Euler Explícito (O1)', alpha=0.4)
plt.plot(tiempo_dias, dist_rk2, 'g-', label='Runge-Kutta O2 (O2)', alpha=0.6)
plt.plot(tiempo_dias, dist_rk4, 'b-', label='Runge-Kutta O4 (O4 - Punto 6)', lw=2)
# Valores de referencia oficiales
plt.axhline(363300, color='black', linestyle='--', alpha=0.4, label='Perigeo Teórico (~363.300
km)')
plt.axhline(405500, color='black', linestyle=':', alpha=0.4, label='Apogeo Teórico (~405.500
km)')
plt.xlabel('Tiempo (días)')
plt.ylabel('Distancia Tierra-Luna (km)')
plt.title('Estabilidad Orbital a Largo Plazo (180 días)\nComparativa: Euler vs RK2 vs RK4')

```

```
plt.legend(loc='upper right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# — Validación de Amplitudes Finales —————
def reportar_extremos(distancias, nombre):
    print(f"\n[{nombre}] tras {dias_simulacion} días:")
    print(f"  Mínima distancia alcanzada (Perigeo): {distancias.min():.1f} km")
    print(f"  Máxima distancia alcanzada (Apogeo) : {distancias.max():.1f} km")

reportar_extremos(dist_euler, "Euler Explícito")
reportar_extremos(dist_rk2, "Runge-Kutta O2")
reportar_extremos(dist_rk4, "Runge-Kutta O4")
```

Anexo II

Output de los scripts

Salida — E1.py: Validación Órbita Lunar

```
=====
VALIDACIÓN – Euler Explícito
=====
Perigeo simulado : 363300.0 km (esperado ≈ 363300 km)
Apogeo simulado : 435505.7 km (esperado ≈ 405500 km)
Vel. máxima      : 1.0763 km/s (esperado ≈ 1.076 km/s)
Vel. mínima      : 0.9276 km/s (esperado ≈ 0.963 km/s)

=====
VALIDACIÓN – Runge-Kutta 02
=====
Perigeo simulado : 363300.0 km (esperado ≈ 363300 km)
Apogeo simulado : 405908.8 km (esperado ≈ 405500 km)
Vel. máxima      : 1.0760 km/s (esperado ≈ 1.076 km/s)
Vel. mínima      : 0.9631 km/s (esperado ≈ 0.963 km/s)
```

Salida — 2.py: Condiciones Iniciales de Orion

```
Datos Cargados
Total de filas: 3262
Primer tiempo: 2026-04-02T01:57:37,084
Último tiempo: 2026-04-10T23:53:16,723

Rango entre las 4am a 6am del 3 de abril
Filas disponibles : 30
```

	tiempo	x_km	y_km	vx_kms	vy_kms
0	2026-04-03T04:02:48,802	-47582.484359	-39794.760136	-1.429465	-2.524611
1	2026-04-03T04:06:48,802	-47923.669167	-40399.084264	-1.413809	-2.511462
2	2026-04-03T04:10:48,802	-48261.143935	-41000.280293	-1.398545	-2.498542
3	2026-04-03T04:14:48,802	-48595.001082	-41598.402366	-1.383659	-2.485845
4	2026-04-03T04:19:39,017	-48994.005198	-42317.637936	-1.366141	-2.470777
5	2026-04-03T04:23:39,017	-49320.179659	-42909.152801	-1.352036	-2.458547
6	2026-04-03T04:27:39,017	-49643.009020	-43497.756708	-1.338263	-2.446518
7	2026-04-03T04:31:39,017	-49962.571384	-44083.497335	-1.324809	-2.434686
8	2026-04-03T04:35:39,017	-50278.941912	-44666.420983	-1.311662	-2.423043
9	2026-04-03T04:39:39,017	-50592.192966	-45246.572632	-1.298812	-2.411585
10	2026-04-03T04:43:39,017	-50902.394262	-45823.995994	-1.286246	-2.400306
11	2026-04-03T04:47:39,017	-51209.612990	-46398.733562	-1.273955	-2.389202
12	2026-04-03T04:51:39,017	-51513.913949	-46970.826660	-1.261930	-2.378268
13	2026-04-03T04:55:39,017	-51815.359657	-47540.315484	-1.250160	-2.367499
14	2026-04-03T04:59:39,017	-52114.010463	-48107.239147	-1.238637	-2.356891
15	2026-04-03T05:03:39,017	-52409.924647	-48671.635720	-1.227353	-2.346439
16	2026-04-03T05:07:39,017	-52703.158522	-49233.542271	-1.216300	-2.336140
17	2026-04-03T05:11:39,017	-52993.766519	-49792.994899	-1.205470	-2.325989
18	2026-04-03T05:15:39,017	-53281.801278	-50350.028774	-1.194855	-2.315983
19	2026-04-03T05:19:39,017	-53567.313723	-50904.678169	-1.184449	-2.306118
20	2026-04-03T05:23:39,017	-53850.353143	-51456.976490	-1.174246	-2.296390
21	2026-04-03T05:27:39,017	-54130.967264	-52006.956308	-1.164238	-2.286797
22	2026-04-03T05:31:39,017	-54409.202313	-52554.649391	-1.154419	-2.277334
23	2026-04-03T05:35:39,017	-54685.103085	-53100.086725	-1.144784	-2.267998
24	2026-04-03T05:39:39,017	-54958.713002	-53643.298551	-1.135328	-2.258787
25	2026-04-03T05:43:39,017	-55230.074174	-54184.314379	-1.126044	-2.249698
26	2026-04-03T05:47:39,017	-55499.227451	-54723.163024	-1.116928	-2.240727
27	2026-04-03T05:51:39,017	-55766.212472	-55259.872620	-1.107974	-2.231872
28	2026-04-03T05:55:39,017	-56031.067720	-55794.470648	-1.099179	-2.223130
29	2026-04-03T05:59:39,017	-56293.830564	-56326.983954	-1.090537	-2.214499

```
Condiciones iniciales elegidas
tiempo: 2026-04-03T04:02:48,802
x0: -47582484.36 m (-47582.5 km)
y0: -39794760.14 m (-39794.8 km)
vx0: -1429.4650 m/s (-1.4295 km/s)
vy0: -2524.6110 m/s (-2.5246 km/s)

Distancia a la Tierra Inicial: 62029958.50 m (62030.0 km)
Velocidad total: 2901.2120 m/s (2.9012 km/s)

Vector u0 para integrador
u0 = [-4.75824844e+07 -3.97947601e+07 -1.42946502e+03 -2.52461104e+03]
```

Salida — 3.py: Trayectoria de Orion

```
Flyby detectado: 2026-04-06 23:02:51.667000+00:00 | ángulo 2D = -111.03°  
Luna en flyby : (-137974, -358785) km
```

```
Condiciones iniciales Orion: 2026-04-03 04:02:48.802000+00:00
```

```
Posición : (-47582.5, -39794.8) km
```

```
Velocidad : (-1429.47, -2524.61) m/s
```

```
Luna en t0 (posición real): (-363454, -125158) km
```

```
Pasos Orion: 11270 (7.83 días)
```

```
Euler: reentrada en paso 7402 (123.4 h)
```

```
RK2: reentrada en paso 7131 (118.8 h)
```

```
Gráfico guardado: punto3_trayectoria_orion.png
```

```
=====
COMPARACIÓN FINAL
=====
```

Euler:

```
Pasos integrados      : 7403
```

```
Distancia mín. a Luna : 413 km
```

```
Distancia final Tierra: 6181 km
```

RK2:

```
Pasos integrados      : 7132
```

```
Distancia mín. a Luna : 224 km
```

```
Distancia final Tierra: 6416 km
```

```
Simulando 180.0 días con un paso h = 1.0 hora...
```

```
[Euler Explícito] tras 180.0 días:
```

```
  Mínima distancia alcanzada (Perigeo): 363300.0 km
```

```
  Máxima distancia alcanzada (Apogeo) : 620218.1 km
```

```
[Runge-Kutta 02] tras 180.0 días:
```

```
  Mínima distancia alcanzada (Perigeo): 363300.0 km
```

```
  Máxima distancia alcanzada (Apogeo) : 405917.1 km
```

```
[Runge-Kutta 04] tras 180.0 días:
```

```
  Mínima distancia alcanzada (Perigeo): 363300.0 km
```

```
  Máxima distancia alcanzada (Apogeo) : 405848.9 km
```

Gráficos

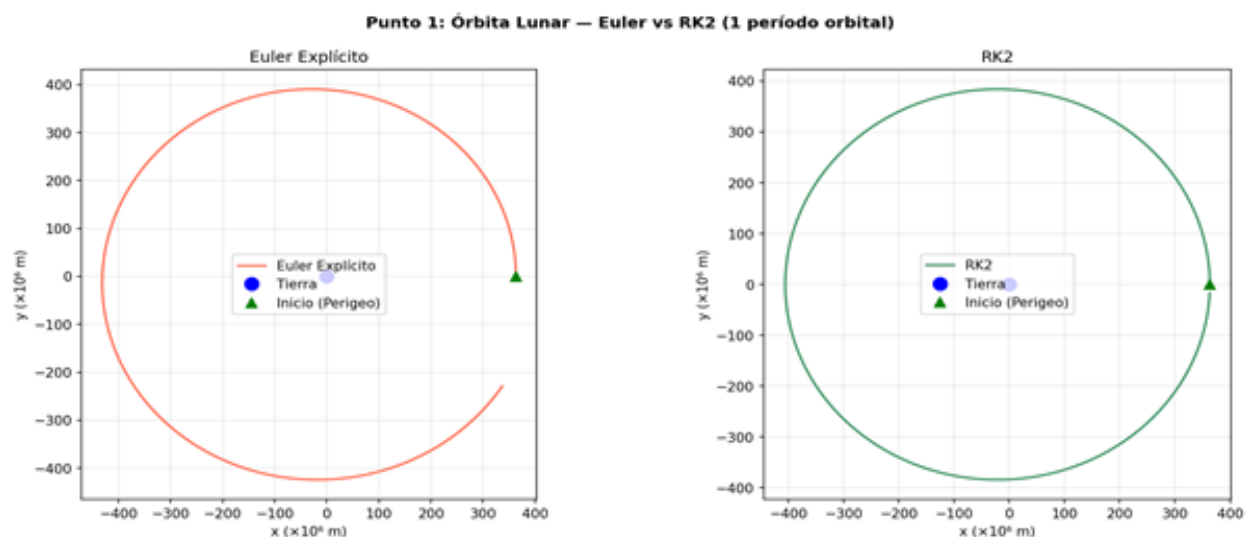
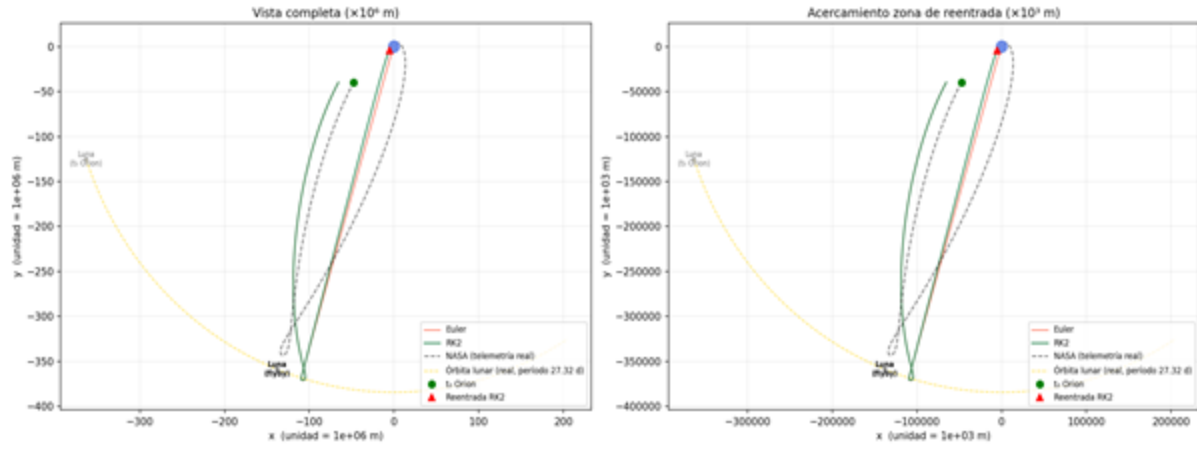


Figura 1: Órbita lunar simulada con Euler (izq.) y RK2 (der.) durante un período orbital de 27,3 días.

Punto 3: Trayectoria Orion — Artemis II
Euler vs RK2 vs Telemetría NASA | Luna en posición real (derivada del flyby)



Punto 3: Trayectoria Orion — Artemis II
Euler vs RK2 vs Telemetría NASA | Luna en posición real (derivada del flyby)

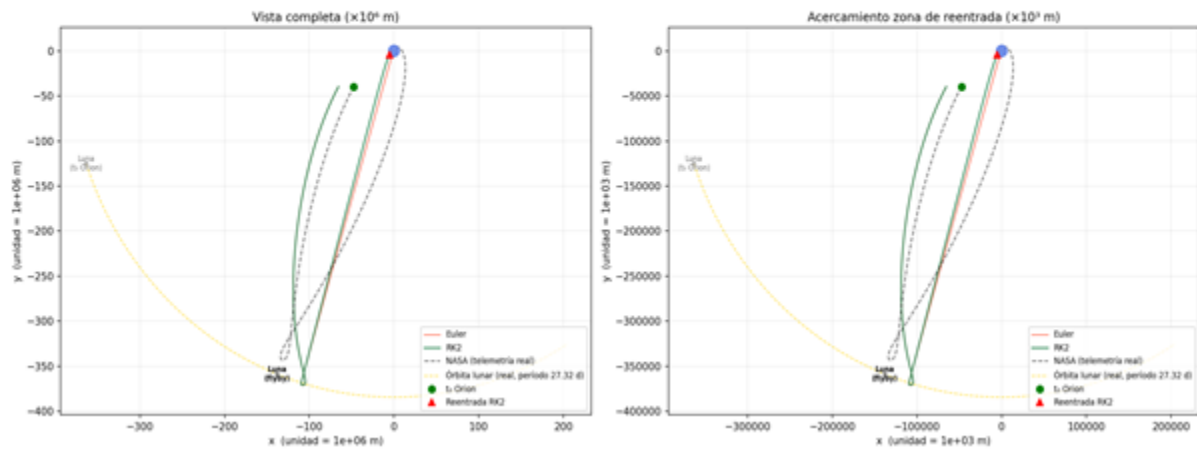


Figura A2: Trayectoria Orion — Euler vs RK2 vs telemetría NASA. [Autores: Tkaczyk 111517 | Fullana 110247]

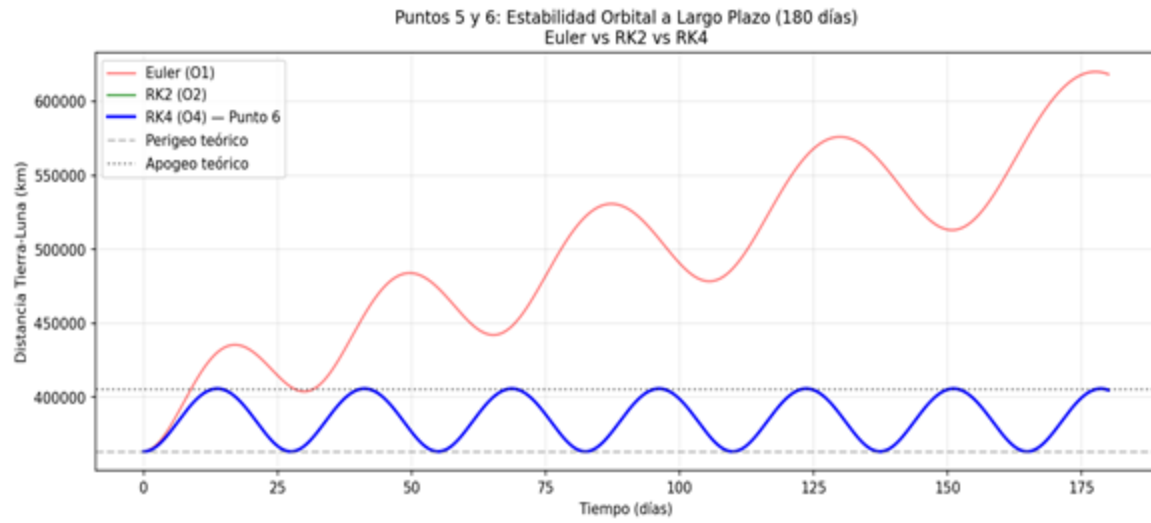


Figura A3: Estabilidad orbital 180 días — Euler vs RK2 vs RK4. [Autores: Tkaczyk 111517 | Fullana 110247]